

Visualizing Data

As a general-purpose programming language, Python is incredibly useful for analyzing data and visualizing results. This activity is a first look at matplotlib, one of the most widely used 2D plotting libraries.

Manager:

Recorder:

Presenter:

Reflector:

Content Learning Objectives

After completing this activity, students should be able to:

- Explain the basic structure of code for plotting a mathematical function.
- Analyze visually the behavior of the Python random number generator.
- Read data from a CSV file and generate histograms of various columns.

Process Skill Goals

During the activity, students should make progress toward:

- Navigating the documentation for a third-party library. (Information Processing)



Copyright © 2021 C. Mayfield and T. Shepherd. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Model 1 Simple Plot

When analyzing data, it's helpful to create charts, plots, and other visualizations. Doing so allows you to see important numerical relationships. Enter the following code into a Python Editor, and run the program.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 def model_one():
5     x = np.arange(0.0, 2.0, .01)
6     y = np.sin(2 * np.pi * x)
7     plt.plot(x, y)
8     plt.xlabel('time (s)')
9     plt.ylabel('volts (mV)')
10    plt.show()
11
12 if __name__ == "__main__":
13     model_one()
```

Questions (15 min)

Start time:

1. Identify in the source code which line numbers:

a) generated the data?

c) displayed the window?

b) set the axes properties?

d) plotted the actual data?

2. Describe in your own words what is being plotted.

A sine wave: $y = \sin(2\pi x)$ for x from 0.0 up to (not including) 2.0 in steps of 0.01, i.e. 200 points. Because the frequency is 1 cycle per unit of x , the plot shows two full cycles of a 1 Hz wave, with time in seconds on the x axis and voltage in millivolts on the y axis.

3. Modify the code to plot only one cycle of the sine wave (instead of two). Write the edited line of code below.

```
x = np.arange(0.0, 1.0, .01)
```

4. Change the third argument of `np.arange` from 0.01 to 0.15. What is the result?

Only 14 points are generated (0, 0.15, 0.30, ... 1.95), so the curve is drawn with straight line segments between far-apart samples and looks jagged/angular instead of a smooth sine wave.

5. Add "o" as a third argument to the plot function. What is the result?

`plt.plot(x, y, 'o')` draws each data point as a circle marker with no connecting line, so the sine wave appears as a series of dots (200 of them with step 0.01; only 14 with step 0.15).

6. How does the third parameter of `np.arange` affect how the plot looks?

It is the step (spacing) between consecutive x values, so it controls how many points are sampled: a small step (0.01) gives many points and a smooth curve; a large step (0.15) gives few points and a coarse, jagged plot.

7. How would you modify the code to plot the function $y = x^2 - 1$ instead? Show the results from -2 to +2.

```
x = np.arange(-2.0, 2.01, .01) # -2 to +2 inclusive
y = x ** 2 - 1
plt.plot(x, y); plt.xlabel('x'); plt.ylabel('y = x^2 - 1'); plt.show()
```

8. Which two Python libraries are used in Model 1? Quickly search the Internet and find their websites. Write a one-sentence description about each library. (Can you identify the third library?)

matplotlib (<https://matplotlib.org>) - a comprehensive library for creating static, animated and interactive 2D plots and charts in Python. NumPy (<https://numpy.org>) - the fundamental package for numerical computing in Python, providing fast N-dimensional arrays and math functions such as `np.arange`, `np.sin` and `np.pi`. The third library is the Python standard library / built-ins (and `matplotlib.pyplot` is itself the sub-module that gives the MATLAB-style plotting interface).

Model 2 Histograms

You can generate a sequence of numbers using the `random` module. Merge the code below into your program from Model 1. Run the program, and view the output.

```
1 import matplotlib.pyplot as plt
2 import random
3
4 def model_two(npts):
5     numbers = []
6     for _ in range(npts):
7         numbers.append(random.random())
8     plt.hist(numbers)
9     plt.show()
10
11 if __name__ == "__main__":
12     model_two(100)
```

Questions (10 min)

Start time:

9. Based on the Python code:

a) What is the range of values generated by the random function?

b) How many random values are generated?

10. Based on the figure plotted:

a) How many bars are displayed?

b) What is the width of each bar?

c) What is the sum of the heights of the bars?

11. Based on your answers above, what are appropriate labels for the x and y axes?

x axis: 'random value' (the value of the number, 0 to 1, split into bins); y axis: 'count' / 'frequency' (how many of the 100 random numbers fall into each bin).

12. Increase the argument of `model_two` to 1000, 10000, and 100000. Describe how the output plot changes when you run the program.

The bars get taller (their heights sum to n) and much more even in height. With 100 numbers the bars vary a lot (e.g. 5 to 16); with 1000 they are roughly 100 each; with 10000 and 100000 the histogram is nearly flat, showing that `random.random()` is uniform on $[0, 1)$.

13. Add the number 50 as second argument to the `hist` function. What is the meaning of the result?

The second argument is the number of bins: the range 0-1 is split into 50 equal-width bins (width 0.02) instead of 10, so 50 narrower bars are drawn and each bar's height is the count of numbers in that 0.02-wide interval (about 2 each for $n=100$, so the plot looks much noisier).

14. In general, describe what the `hist` function does with the list of random numbers to create this type of plot.

`hist()` finds the min and max of the list, divides that range into equal-width intervals (bins, 10 by default), counts how many values fall into each interval, and draws one bar per bin whose height is that count. It returns the counts and the bin edges.

Model 3 CSV Data

Recall that “Comma Separated Values” is a common file format when exporting data from spreadsheets and databases. Each line of the file is a row, and each column is separated by a comma. Cells that contain commas are wrapped in quote marks.

data.csv file contents:

```
Name,Location,URL,Students
Westminster College,"Salt Lake City, UT",westminstercollege.edu,2135
Muhlenberg College,"Allentown, PA",muhlenberg.edu,2330
University of Maine,"Orono, ME",umaine.edu,8677
James Madison University,"Harrisonburg, VA",jmu.edu,19019
Michigan State University,"East Lansing, MI",msu.edu,38853
```

Python includes a csv module (<https://docs.python.org/3/library/csv.html>) that makes it easy to read and write CSV files.

```
import csv
```

```
infile = open("data.csv")
data = csv.reader(infile)
names = next(data) # column names
for row in data:
    print(row[1]) # 2nd column
```

Program output:

```
Salt Lake City, UT
Allentown, PA
Orono, ME
Harrisonburg, VA
East Lansing, MI
```

Questions (20 min)

Start time:

15. In the example data.csv file above:

- a) In what way is the first line different?
- b) How many rows of data are there? How many columns?

16. Compare data.csv with the program output:

- a) Are quote marks included in data.csv? In the program output?
- b) What is the purpose of the quote marks?

17. In the Python code above:

- a) Which line of code reads the first line of the file?
- b) What type of data does the variable row contain?

In 2013, the U.S. Department of Education released the “College Scorecard” website to help students and families compare institutions of higher education. The Scorecard data includes information like average cost of attendance, graduation and retention rates, student body demographics, etc.

18. Download the “Most Recent Institution-Level Data” from <https://collegescorecard.ed.gov/data/> (listed halfway down on the right). Open the CSV file in Excel or a similar program, and skim its contents. 

a) How many rows does it have?

b) How many columns does it have?

19. Column KE is named UGDS, which means “Enrollment of undergraduate certificate / degree-seeking students”.

a) What is the range of values in this column?

b) Which school has the most students enrolled?

c) Do all rows have an integer value for UGDS?

20. Based on the code in Model 2 and Model 3, write a program that plots a histogram of the UGDS column. Complete the following steps to consider each part of the program.

a) What two import statements will you need at the top?

```
import matplotlib.pyplot as plt
import csv
```

b) What three statements prepare the csv file for reading?

```
infile = open('scorecard.csv', newline="", encoding='utf-8')
data = csv.reader(infile)
names = next(data) # skip/keep the header row
```

c) What code is necessary to read the entire column into a list?

```
ugds = []
for row in data:
    ugds.append(row[290]) # 290 == names.index('UGDS'), column KE
```

d) By default, data from text/csv files are read as strings. Write the code to convert the

row[290] values to integers. Be sure not include the "NULL" values in the final list.

```
ugds = []  
for row in data:  
    if row[290] not in ('NULL', 'NA', ''): # skip missing values  
        ugds.append(int(row[290]))
```

e) Write the last two lines that plot and show the histogram.

```
plt.hist(ugds, 50)  
plt.show()
```

21. Run the program, and compare your results with another team's. What does the histogram tell you about undergraduate enrollments in the United States?

The distribution is extremely right-skewed. Almost all institutions are small: the very first bar (0 to ~3,000 students) holds the vast majority of the 5,478 schools, and the bars fall off so quickly that schools above ~20,000 are almost invisible; only a handful of giant (mostly online) universities such as SNHU (163k), WGU (155k) and Univ. of Phoenix (86k) form a long thin tail. A log-scaled y axis or a cap on the x range is needed to see the typical school size.

22. What other questions could you ask about these data? How would you answer them using histograms, line charts, and scatter plots?

Histograms: distribution of admission rate (ADM_RATE), average SAT (SAT_AVG), cost of attendance, or graduation rate; compare public vs private (CONTROL) by overlaying two histograms. Scatter plots: does cost relate to graduation rate? admission rate vs SAT_AVG? enrollment vs median earnings? Line charts: sort schools by size and plot cumulative enrollment share, or plot average cost by state (STABBR) in order, or trends over years if multiple yearly files are combined.